AUTOCORRECT USING SEQ2SEQ MODEL

Deep Learning Project Report

Technology Stack

TensorFlow | Keras | LSTM | Python | Streamlit

CH.SAIPAVAN REDDY,K.VISHWANATH

23STUCHH010483,23STUCHH010520

1. Problem Definition & Goal Setting

1.1 Problem Statement

Typing errors are a pervasive challenge in digital text input. These errors arise from human mistakes such as spelling errors, missing characters, transpositions, or incorrect word formation. Traditional rule-based spell checkers rely on dictionary lookups and fixed correction rules, making them brittle when faced with complex, out-of-vocabulary, or context-dependent errors. They are unable to learn patterns from data and often produce no correction or incorrect suggestions for novel misspellings.

1.2 Proposed Solution

This project addresses the limitations of rule-based spell checkers by building an intelligent autocorrect system using a Sequence-to-Sequence (Seq2Seq) deep learning model. The model learns a direct mapping between misspelled character sequences and their corrected forms, enabling it to generalize across a wide variety of spelling errors.

1.3 Goals & Objectives

- Learn a character-level mapping between incorrect and correct word sequences
- Handle real-world typing errors including substitutions and transpositions
- Provide intelligent autocorrection comparable to modern keyboard behavior
- Deploy the trained model in a Streamlit web application for interactive use

2. Data Acquisition & Preparation

2.1 Data Source

The dataset used for training is derived from a large multilingual translation corpus (fra.txt). English sentences are extracted and used as the source of clean, correctly spelled text. Misspelled variants are synthetically generated by introducing controlled noise into these clean sentences.

2.2 Noise Injection Strategy

For each clean sentence, up to 5 noisy variants are generated. With a 90% probability, one character in the sentence is randomly replaced with a random ASCII character, simulating a realistic typing error. This yields a large and diverse training dataset of (misspelled, correct) pairs.

Parameter	Value
Total Samples	120,000
Noise Type	Random character substitution
Noise Probability	90% per augmentation
Augmentations per sentence	Up to 5

2.3 Preprocessing Steps

1. Lowercasing: all input text is converted to lowercase
2. Special character removal: non-alphabetic characters are removed using regex
3. Character tokenization: each unique character is mapped to an integer index
4. One-hot encoding: characters are represented as sparse binary vectors
5. Sequence padding: sequences are zero-padded to the maximum length
6. Train/Validation split: 80% training, 20% validation

2.4 Vocabulary Statistics

The character vocabularies for input and output are built from the entire dataset:

Statistic	Value
Unique encoder tokens (input vocab)	~29 characters
Unique decoder tokens (output vocab)	~30 characters
Max encoder sequence length	Variable (sentence-dependent)

Statistic	Value
Max decoder sequence length	Variable + 2 (start/stop tokens)

3. Model Building & Architecture

3.1 Model Type

The model follows a classic Sequence-to-Sequence (Seq2Seq) architecture with an Encoder-Decoder structure. This paradigm is well-suited for tasks where the input and output sequences may differ in length and content, such as translation or spell correction.

3.2 Architecture Overview

```
Misspelled Input  →  [Encoder LSTM]  →  Context Vector  →  [Decoder LSTM]  →
                                        Corrected Output
```

3.3 Encoder

The encoder processes the misspelled input sequence character by character:

- Input layer accepts one-hot encoded character vectors of shape (None, num_encoder_tokens)
- A single LSTM layer with 128 latent units processes the full sequence
- Dropout of 0.4 is applied to prevent overfitting
- The encoder discards its output sequence and retains only the final hidden state (state_h) and cell state (state_c), forming the context vector

3.4 Decoder

The decoder generates the corrected output sequence one character at a time:

- Input: one-hot encoded target sequence (shifted by one position — teacher forcing)
- A single LSTM layer with 128 latent units, initialized with the encoder states
- Dropout of 0.4 is applied
- A Dense layer with softmax activation produces a probability distribution over the target vocabulary at each time step

3.5 Inference Models

At inference time, the single training model is split into two separate models to support auto-regressive decoding:

- Encoder Model: maps input sequence to context vector (state_h, state_c)
- Decoder Model: takes a single character + current states, produces next character + new states

Component	Layer Type	Units	Dropout
Encoder Input	Input Layer	num_encoder_tokens	-
Encoder	LSTM	128	0.4
Decoder Input	Input Layer	num_decoder_tokens	-
Decoder	LSTM	128	0.4
Output	Dense (softmax)	num_decoder_tokens	-

4. Model Compilation & Training

4.1 Compilation Settings

Parameter	Value
Optimizer	Adam (default learning rate)
Loss Function	Categorical Crossentropy
Metrics	Accuracy
Batch Size	64
Epochs	15
Validation Split	20%

4.2 Training Strategy: Teacher Forcing

During training, teacher forcing is applied: at each decoding step, the ground-truth character from the target sequence (shifted by one position) is fed into the decoder, rather than its own previous prediction. This significantly accelerates convergence and stabilizes training, though at inference time the model must rely on its own predictions.

4.3 Training Observations

- Loss steadily decreases across epochs
- Accuracy improves progressively on both training and validation sets
- Dropout regularization at rate 0.4 mitigates overfitting on the large synthetic dataset
- The trained model is saved as auto.h5 and character vocabulary mappings are persisted in var.pkl

5. Evaluation & Optimization

5.1 Evaluation Methodology

The model is evaluated on held-out sequences not seen during training. The decoded output is compared against the original (correct) text using exact sequence match and character-level accuracy.

5.2 Sample Predictions

Input (Misspelled)	Predicted Output	Actual (Correct)	Match
speling	spelling	spelling	Yes
wrld	world	world	Yes
ygu mqst leave now	you must leave now	you must leave now	Yes
helo wrold	hello world	hello world	Yes

5.3 Optimization Techniques Applied

- Dropout (rate 0.4) on both LSTM layers to prevent overfitting
- Synthetic data augmentation to create a diverse training corpus
- Teacher forcing during training for fast and stable convergence
- Adam optimizer with adaptive learning rate for efficient gradient descent

5.4 Known Challenges & Limitations

- Long sequences: the fixed-size context vector may not capture all information from very long inputs
- Unseen error patterns: generalization to errors not represented in the synthetic training data may be limited
- Character-only vocabulary: out-of-vocabulary characters (digits, punctuation) are stripped during preprocessing
- No attention mechanism: a future improvement would be adding Bahdanau or Luong attention

6. Deployment using Streamlit

6.1 Deployment Platform

The trained model is deployed as a lightweight interactive web application using Streamlit. Streamlit provides a simple Python-native framework for building data science applications without requiring any frontend expertise. The app allows users to enter misspelled text and receive the corrected output in real time.

6.2 Application Architecture

- Model Loading: the trained Keras model (auto.h5) and vocabulary mappings (var.pkl) are loaded once at startup using @st.cache_resource for efficiency
- Encoder & Decoder Models: inference models are reconstructed from the saved training model
- User Interface: a clean text input field, a correction button, and a result display area
- Prediction Pipeline: user input is preprocessed, one-hot encoded, passed through the encoder, then decoded character-by-character

6.3 Running the Application

To launch the Streamlit app locally:

```
# Install dependencies
pip install streamlit tensorflow numpy
streamlit run app.py
```

6.4 Monitoring & Maintenance

- Track prediction accuracy on real user inputs collected through the app
- Periodically retrain the model with new real-world error examples
- Handle edge cases such as empty input, very long sequences, or unsupported characters
- Monitor response latency and optimize model loading for production scale

Conclusion

This project successfully demonstrates the application of deep learning — specifically the Sequence-to-Sequence (Seq2Seq) architecture — to the problem of automatic spell correction. By training a character-level LSTM encoder-decoder model on 120,000 synthetically augmented sentence pairs, the system learns to map misspelled inputs to their correct forms without any handcrafted rules.

Unlike traditional dictionary-based spell checkers, this model generalizes from data, making it robust to a wide variety of typing errors. The model is deployed as an interactive Streamlit web application, providing an intuitive interface for real-time autocorrection.

Key takeaways from this project:

- Seq2Seq models are highly effective for character-level sequence correction tasks
- Synthetic noise injection is a powerful strategy for generating large, diverse training data
- Teacher forcing enables fast and stable training of auto-regressive models
- Streamlit provides a rapid and effective path from trained model to deployed application

Future enhancements could include adding an attention mechanism, training on larger and more diverse corpora, and fine-tuning on domain-specific text for improved performance in specialized contexts.